

a)Rate monotonic scheduling (RMS) Algorithm

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int gcd(int a, int b) {
```

```
    while (b != 0) {
```

```
        int temp = b;
```

```
        b = a % b;
```

```
        a = temp;
```

```
    }
```

```
    return a;
```

```
}
```

```
int lcm(int a, int b) {
```

```
    return (a * b) / gcd(a, b);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of processes:");
```

```
    scanf("%d", &n);
```

```
    int burst[n], period[n];
```

```
printf("Enter the CPU burst times:\n");
```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &burst[i]);  
}
```

```
printf("Enter the time periods:\n");
```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &period[i]);  
}
```

```
int hyper = period[0];
```

```
for (int i = 1; i < n; i++) {  
    hyper = lcm(hyper, period[i]);  
}
```

```
printf("LCM=%d\n\n", hyper);
```

```
printf("Rate Monotone Scheduling:\n");
```

```
printf("PID\tBurst\tPeriod\n");
```

```
for (int i = 0; i < n; i++) {  
    printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);  
}
```

```
double utilization = 0;
```

```
for (int i = 0; i < n; i++) {
```

```

        utilization += (double)burst[i] / period[i];
    }

    double bound = n * (pow(2, (double)1/n) - 1);

    printf("\n%f <= %f => %s\n",
        utilization, bound,
        (utilization <= bound) ? "true" : "false");

    printf("Scheduling occurs for %d ms\n\n", hyper);

    int remaining[n];
    for (int i = 0; i < n; i++) {
        remaining[i] = 0;
    }

    int prev = -2; // track last running process

    for (int time = 0; time < hyper; time++) {

        // Release tasks
        for (int i = 0; i < n; i++) {
            if (time % period[i] == 0) {
                remaining[i] = burst[i];
            }

```

```
}
```

```
int selected = -1;
```

```
for (int i = 0; i < n; i++) {
```

```
    if (remaining[i] > 0) {
```

```
        if (selected == -1 || period[i] < period[selected]) {
```

```
            selected = i;
```

```
        }
```

```
    }
```

```
}
```

```
if (selected != prev) {
```

```
    if (selected == -1)
```

```
        printf("%dms onwards: CPU is idle\n", time);
```

```
    else
```

```
        printf("%dms onwards: Process %d running\n", time, selected + 1);
```

```
    prev = selected;
```

```
}
```

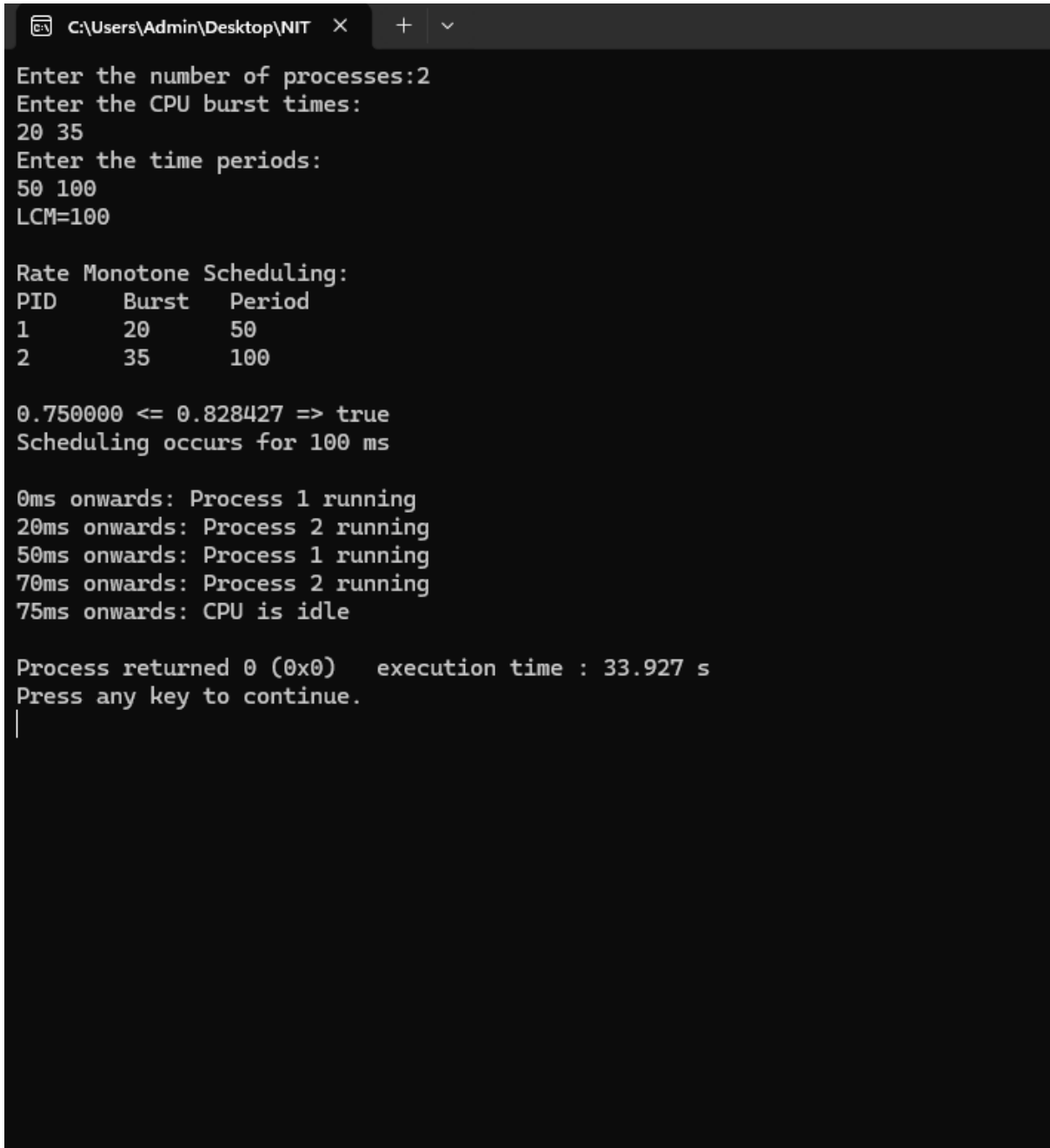
```
if (selected != -1) {
```

```
    remaining[selected]--;
```

```
}
```

```
}
```

```
    return 0;
}
```



```
C:\Users\Admin\Desktop\NIT X + v
Enter the number of processes:2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID      Burst   Period
1         20      50
2         35     100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
100ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 33.927 s
Press any key to continue.
|
```

b) Earliest Deadline first (EDF) pseudocode

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main() {  
    int n = 3;  
    int pid[] = {2, 1, 3};  
    int burst[] = {2, 3, 2};  
    int deadline[] = {4, 7, 8};  
    int period[] = {5, 20, 10};  
  
    int remaining[3];  
  
    for (int i = 0; i < n; i++) {  
        remaining[i] = burst[i];  
    }  
  
    printf("PID\tBurst\tDeadline\tPeriod\n");  
    for (int i = 0; i < n; i++) {  
        printf("%d\t%d\t%d\t\t%d\n", pid[i], burst[i], deadline[i], period[i]);  
    }  
  
    int time = 0;  
    int total_time = 20;  
  
    printf("\nScheduling occurs for %d ms\n\n", total_time);  
  
    while (time < total_time) {
```

```
int selected = -1;

int min_deadline = INT_MAX;

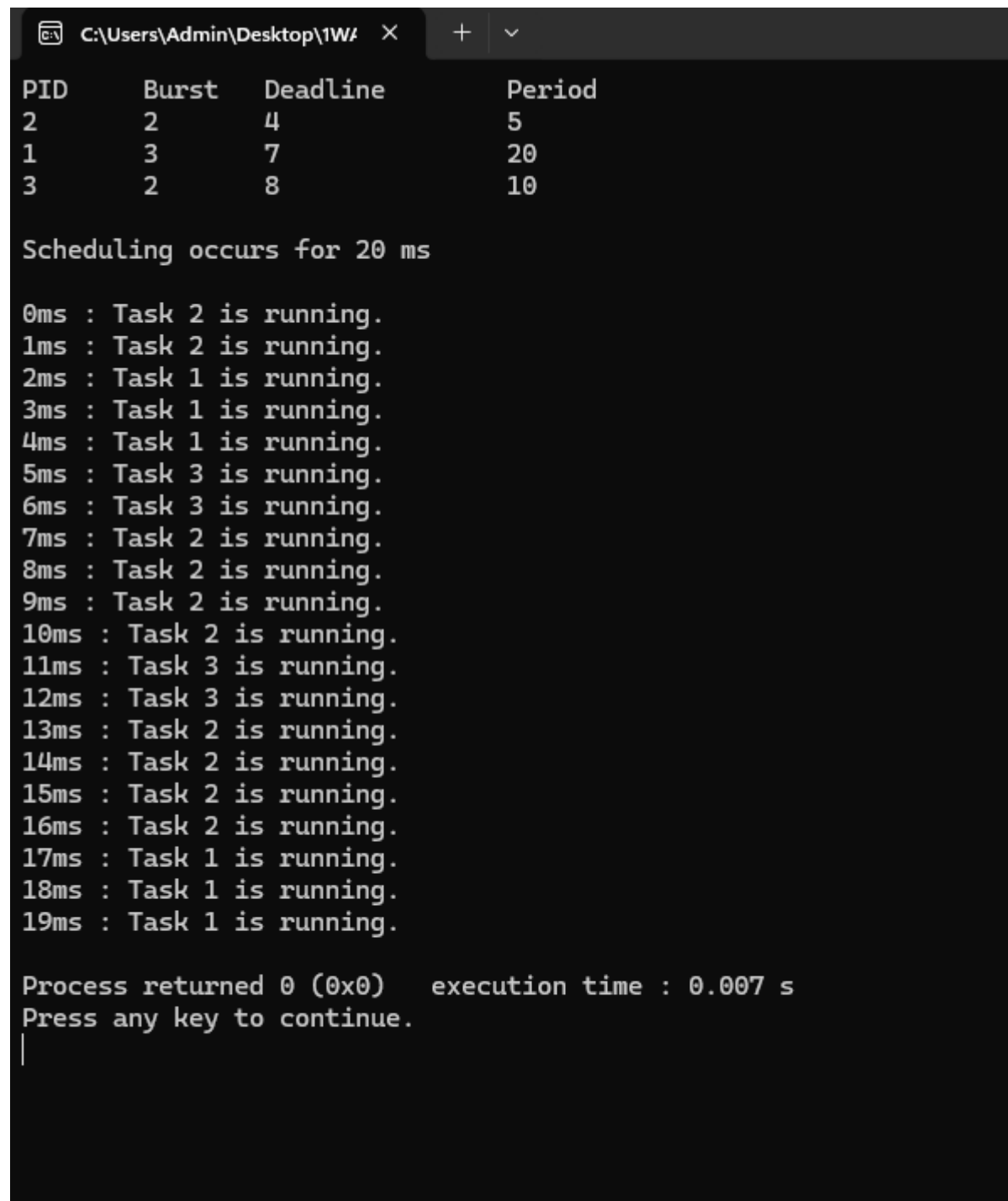
for (int i = 0; i < n; i++) {
    if (remaining[i] > 0) {
        if (deadline[i] < min_deadline) {
            min_deadline = deadline[i];
            selected = i;
        }
    }
}

if (selected == -1) {
    printf("%dms : CPU is idle.\n", time);
} else {
    printf("%dms : Task %d is running.\n", time, pid[selected]);

    remaining[selected]--;

    if (remaining[selected] == 0) {
        remaining[selected] = burst[selected];
        deadline[selected] += period[selected];
    }
}
```

```
        time++;  
    }  
  
    return 0;  
}
```



The screenshot shows a Windows command prompt window with the title bar "C:\Users\Admin\Desktop\1W/ X". The window displays the output of a program, which includes a table of task parameters and a detailed log of task execution over 20 milliseconds.

PID	Burst	Deadline	Period
2	2	4	5
1	3	7	20
3	2	8	10

Scheduling occurs for 20 ms

0ms : Task 2 is running.
1ms : Task 2 is running.
2ms : Task 1 is running.
3ms : Task 1 is running.
4ms : Task 1 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 2 is running.
8ms : Task 2 is running.
9ms : Task 2 is running.
10ms : Task 2 is running.
11ms : Task 3 is running.
12ms : Task 3 is running.
13ms : Task 2 is running.
14ms : Task 2 is running.
15ms : Task 2 is running.
16ms : Task 2 is running.
17ms : Task 1 is running.
18ms : Task 1 is running.
19ms : Task 1 is running.

Process returned 0 (0x0) execution time : 0.007 s
Press any key to continue.
|

c) Proportional share scheduling

```
#include<stdio.h>

#include <stdlib.h>

#include <time.h>

typedef struct
{
    char name[5]; int tickets; }
    Process;

int main() {
    int n, total_tickets = 0; float total_T =0.0;
    printf("Enter the number of Processes: ");
    scanf("%d", &n);

    Process p[n];

    srand(time(NULL));

    for (int i = 0; i < n; i++)
    { printf("\nProcess %d:\n", i + 1);
      sprintf(p[i].name, "P%d", i + 1);
      printf("Tickets: ");
      scanf("%d", &p[i].tickets);
      total_tickets += p[i].tickets;
      total_T +=p[i].tickets;
    }

    printf("\n--- Proportional Share Scheduling ---\n");
    printf("Enter the Time Period for scheduling: ");

    int m;
```

```

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index;
    for (int j = 0; j < n; j++)
    {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets)
        {
            winner_index = j; break;
        }
    }
    printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
        p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
        ((p[i].tickets / total_T) * 100));

} return 0; }

```

OUTPUT--

```
"C:\Users\Admin\Desktop\PR" X + v
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 22, Winner: P2
Tickets picked: 22, Winner: P2
Tickets picked: 20, Winner: P2
Tickets picked: 26, Winner: P2
Tickets picked: 23, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)    execution time : 50.055 s
Press any key to continue.
```